

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Обектно-ориентирано програмиране

#### Класове и обекти

Python има много вградени типове като `int`, `str` и други, които могат да се използват при разработване на програми. Но Python също позволява дефиниране на собствени типове. Класът представлява обект. Конкретната реализация на класа е обект.

Аналог на клас от реалния свят е например човека, той има име, възраст и някакви други характеристики. Човекът може да извършва определени действия - да ходи, да бяга, да мисли и т.н. Или изглежда, който включва набор от характеристики и действия, може да се нарече клас. Конкретното изпълнение на този модел може да варира, например, някои хора имат едно име, други имат друго име. И конкретния човек ще представлява обект от този клас. Класът се дефинира с помощта на ключовата дума `class`:

```
class име_на_класа:
```

```
    атрибути_на_класа
```

```
    методи_на_класа
```

Вътре в класа се дефинират атрибутите, които съхраняват различни характеристики на класа, а методите са функциите на класа. Нека да се създаде елементарен клас:

```
class Person:
```

```
    pass
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

В този случай се дефинира класът `Person`, който условно представлява лице. В този случай в класа не са дефинирани никакви методи или атрибути. Въпреки това, тъй като нещо трябва да бъде дефинирано в него, операторът **pass** се използва като заместител на функционалността на класа. Този израз се използва, когато синтактично е необходимо да се дефинира определен код, но ако това не бъде направено, вместо конкретен код може да се вмъкне оператора **pass**.

След като класът е създаден, обектите от този клас могат да бъдат дефинирани.

**Например:**

```
class Person:
```

```
    pass
```

```
tom = Person()    # дефиниран обект tom
```

```
bob = Person()    # дефиниран обект bob
```

След като класът `Person` е дефиниран, се създават два обекта от класа `Person`, `tom` и `bob`. За създаване на обект се използва специална функция - конструктор, който се извиква от името на класа и който връща обект от класа. Т.е. в случая повикването `Person()` представлява извикване към конструктора. Всеки клас има конструктор по подразбиране без параметри:

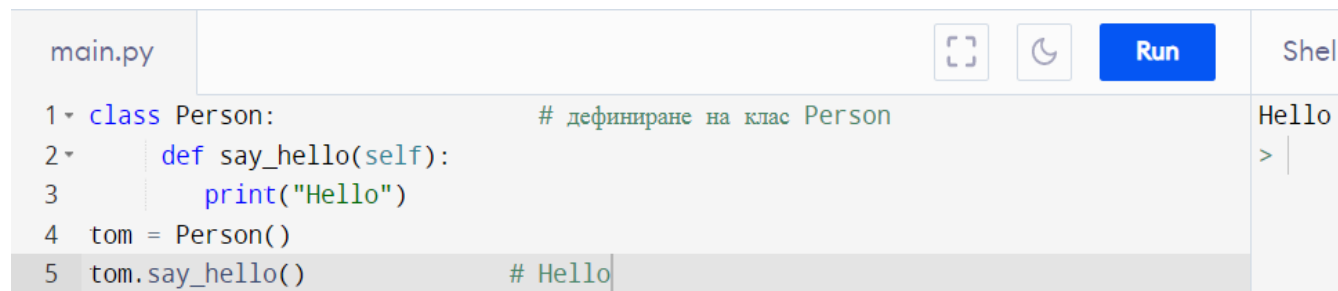
```
tom = Person()    # Person() - извикване на конструктор, който връща обект от класа Person
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Класови методи

Методите на класа всъщност представляват функции, които са дефинирани в класа и които определят поведението му. Например, нека дефинираме клас Person с един метод:

```
class Person:
    def say_hello(self):
        print("Hello")
tom = Person()
tom.say_hello()
```

A screenshot of a Python IDE window titled 'main.py'. The code editor shows a class definition for 'Person' with a 'say\_hello' method that prints 'Hello'. Below the class definition, an instance 'tom' is created and the 'say\_hello' method is called. The output console on the right shows 'Hello' and a prompt '>'. The code is as follows:

```
1 class Person:                                # дефиниране на клас Person
2     def say_hello(self):
3         print("Hello")
4 tom = Person()
5 tom.say_hello()                               # Hello
```

Тук е дефиниран методът say\_hello(), който условно изпълнява Hello и отпечатва низ в конзолата. Когато се дефинират методи на всеки клас, имайте предвид, че всички те трябва да приемат като свой първи параметър препратка към текущия обект, който по конвенция се нарича self. Чрез тази препратка в класа може да се получи достъп до функционалността на текущия обект. Но когато методът се извика, този параметър не се взема предвид. Използвайки името на обект, може да се обърне към неговите методи. За достъп до методите се използва точкова нотация - след името на обекта се поставя точка и след нея идва извикването на метода:

обект.метод([параметри на метода])

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Например, извикване на метод `say_hello()` за показване на поздрав в конзолата:

```
tom.say_hello() # Hello
```

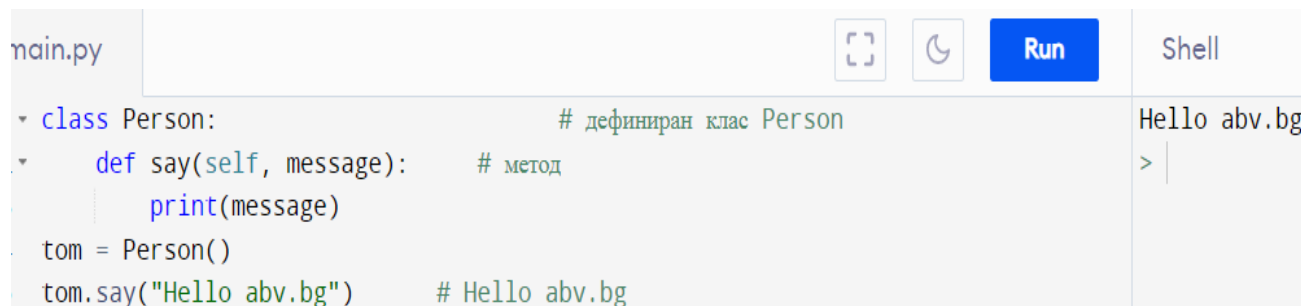
В резултат на това тази програма ще отпечата низа "Hello" в конзолата. Ако трябва методът да приеме други параметри, те се дефинират след параметъра `self` и когато трябва да се извиква такъв метод, трябва да се предадат стойности за тях:

```
class Person:
```

```
    def say(self, message):  
        print(message)
```

```
tom = Person()
```

```
tom.say("Hello abv.bg")
```



The screenshot shows a Python IDE with a file named `main.py`. The code in the editor is as follows:

```
class Person:  
    def say(self, message):  
        print(message)  
tom = Person()  
tom.say("Hello abv.bg")
```

Comments are present: `# дефиниран клас Person` for the class definition, `# метод` for the `say` method, and `# Hello abv.bg` for the final line. The IDE has buttons for `Run` and `Shell`. The `Shell` output shows `Hello abv.bg` and a prompt `> |`.

Методът дефиниран тук е `say()`. Необходими са два параметъра: `self` и `message`. А за втория параметър - `message`, при извикване на метода трябва да предадете стойност. `self`

Чрез ключовата дума `self` може да получите достъп до функционалността на текущия обект в класа:

```
self.атрибут    # обръщение към атрибут  
self.метод     # обръщение към метод
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Например, нека се дефинират два метода в класа Person:

```
class Person:
```

```
    def say(self, message):  
        print(message)
```

```
    def say_hello(self):  
        self.say("Hello work")    # позовавайки се на метода say
```

```
tom = Person()  
tom.say_hello()    # Hello work
```

Тук в един метод - say\_hello() се извиква друг метод - say():

```
self.say("Hello work")
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тъй като методът `say()` приема, в допълнение към `self`, параметри (параметъра `message`), когато методът се извика, се предава стойност за този параметър. Освен това, когато се извиква обектен метод, определено трябва да се използва думата `self`, ако не се използва се извежда грешка:

```
def say_hello(self):  
    say("Hello work")          # Error
```

### Конструктори

За създаване на обект от клас се използва конструктор. Когато се създават обекти от класа `Person`, по-горе, бе използван конструктор по подразбиране, който не приема параметри и на който всички класове имплицитно имат:

```
tom = Person()
```

Въпреки това, може изрично да се дефинира конструктор в класовете, като се използва специален метод, наречен `__init__()` (две тирета от всяка страна).

Например, нека променим класа `Person`, като добавим конструктор към него:

```
class Person:
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
# конструктор
def __init__(self):
    print("Създаване на обект Person")
```

```
def say_hello(self):
    print("Hello")
```

```
tom = Person()    # Създаване на обект в Person
tom.say_hello()   # Hello
```

И така, тук в кода на класа Person са дефинирани конструктор и метод say\_hello(). Като първи параметър конструкторът, подобно на методите, също взема препратка към текущия обект - self. Обикновено конструкторите се използват за дефиниране на действията, които трябва да се предприемат при създаване на обект.

Сега, когато създавате обект:

tom = Person(), ще бъде извикан конструктор от класа Person \_\_init\_\_(), който ще отпечата низа "Създаване на обект Person" в конзолата.

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Атрибути на обекта

Атрибутите съхраняват състоянието на обект. Можете да използвате думата `self`, за да дефинирате и зададете атрибути в рамките на клас. Например, нека дефинираме следния клас `Person`:

```
class Person:
```

```
    def __init__(self, name):
        self.name = name # име
        self.age = 1      # възраст
```

```
tom = Person("Tom")
```

```
# обръщение към атрибутите и получаване на стойност
```

```
print(tom.name)      # Tom
```

```
print(tom.age)       # 1
```

```
# промяна на стойност
```

```
tom.age = 37
```

```
print(tom.age)       # 37
```



## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Сега конструкторът на класа Person приема още един параметър - name. Чрез този параметър името на създаденото лице ще бъде предадено на конструктора. В конструктора се задават два атрибута - name и age (условно името и възрастта на човек):

```
def __init__(self, name):  
    self.name = name  
    self.age = 1
```

Атрибутът self.name е зададен като стойност на променливата name. Атрибутът age е зададен да е 1.

**Ако сме дефинирали конструктор \_\_init\_\_ в клас, вече няма да може да се извика конструктора по подразбиране. Сега трябва да се извика изрично дефинираният конструктор \_\_init\_\_, на който трябва да бъде предадена стойност за параметъра name:**

```
tom = Person("Tom")
```

Освен това, чрез името на обекта, може да се получи достъп до атрибутите на обекта - да се получат и променят стойностите им:

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
print(tom.name)      # получаване на стойността на атрибута name
tom.age = 37         # промяна на стойността на атрибута age
```

По принцип не е необходимо дефиниране на атрибути вътре в класа - Python позволява това да става динамично извън кода:

```
class Person:
```

```
    def __init__(self, name):
        self.name = name # име
        self.age = 1      # възраст
```

```
tom = Person("Tom")
tom.company = "Microsoft"
print(tom.company)    # Microsoft
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тук динамично се задава атрибутът на фирмата, който съхранява работното място на лицето. И след настройка може да се получи и стойността му. В същото време такова определение е изпълнено с грешки. Например, ако се направи опит да се осъществи достъп до атрибут, преди да е дефиниран, програмата ще генерира грешка:

```
tom = Person("Tom")
print(tom.company) # ! Error - AttributeError: Person object has no attribute company
```

За достъп до атрибути на обект вътре в клас, думата `self` също се използва в методите му:

```
class Person:

    def __init__(self, name):
        self.name = name    # име
        self.age = 1        # възраст

    def display_info(self):
        print(f'Name: {self.name} Age: {self.age}')

tom = Person("Tom")
tom.display_info()        # Name: Tom Age: 1
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Това дефинира метода `display_info()`, който отпечатва информация в конзолата. За достъп до атрибутите на обекта в метода се използва думата `self`: `self.name` и `self.age`

### Създаване на обекти

```
class Person:
```

```
    def __init__(self, name):
        self.name = name      # име
        self.age = 1          # възраст

    def display_info(self):
        print(f'Name: {self.name} Age: {self.age}')
```

```
tom = Person("Tom")
tom.age = 37
tom.display_info()      # Name: Tom Age: 37
```

```
bob = Person("Bob")
bob.age = 41
bob.display_info()      # Name: Bob Age: 41
```

## **Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект**

Тук се създават два обекта от класа Person: tom и bob. Те съвпадат с дефиницията на класа Person, имат същия набор от атрибути и методи, но състоянието им е различно. Когато се изпълнява програма, Python динамично ще определи self - той представлява обекта, върху който е извикан методът.

### **Например:**

```
tom.display_info()    # Name: Tom Age: 37
```

Това ще бъде обектът Tom. И когато се извика:  
bob.display\_info()

Това ще бъде обектът Bob. В резултат на това ще се получи следния конзолен изход:

```
Name: Tom Age: 37
```

```
Name: Bob Age: 41
```

### **Капсулиране, атрибути и свойства**

По подразбиране атрибутите в класовете са публични, което означава, че от всяко място в програмата може да се получи атрибут на обект, който може да бъде променен.

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

**Например:**

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name           # установяване на име
```

```
        self.age = 1              # установяване на възраст
```

```
    def display_info(self):
```

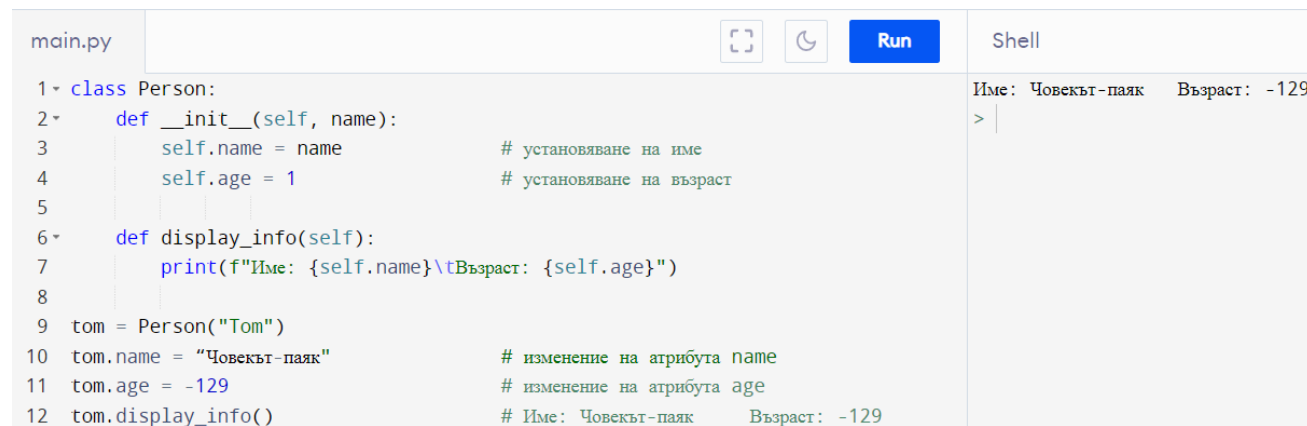
```
        print(f"Име: {self.name}\tВъзраст: {self.age}")
```

```
tom = Person("Tom")
```

```
tom.name = "Човекът-паяк"      # изменение на атрибута name
```

```
tom.age = -129                  # изменение на атрибута age
```

```
tom.display_info()              # Име: Човекът-паяк    Възраст: -129
```



```
main.py  Run  Shell
1 class Person:
2     def __init__(self, name):
3         self.name = name           # установяване на име
4         self.age = 1              # установяване на възраст
5
6     def display_info(self):
7         print(f"Име: {self.name}\tВъзраст: {self.age}")
8
9 tom = Person("Tom")
10 tom.name = "Човекът-паяк"        # изменение на атрибута name
11 tom.age = -129                   # изменение на атрибута age
12 tom.display_info()               # Име: Човекът-паяк    Възраст: -129
```

Име: Човекът-паяк Възраст: -129

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

В този случай може да се присвои неправилна стойност на възрастта или името на човек, например да посочим отрицателна възраст. Такова поведение е нежелателно, така че възниква въпросът за контролиране на достъпа до атрибутите на обекта. Концепцията за капсулиране е тясно свързана с този проблем.

**Капсулирането** е и основната концепция в ООП. Той предотвратява директния достъп до атрибутите на обекта от извикващият код. По отношение на капсулирането директно в езика за програмиране Python, може да скриете атрибутите на класа, като ги направите частни и ограничите достъпа до тях чрез специални методи, които също се наричат **свойства**.

Нека се промени дефинирания клас от предходния слайд, като се дефинират свойства в него:

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
class Person:
    def __init__(self, name):
        self.__name = name        # установяване на име
        self.__age = 1           # установяване на възраст
```

```
    def set_age(self, age):
        if 1 < age < 110:
            self.__age = age
        else:
            print("Недопустима възраст")
```

```
    def get_age(self):
```

```
        return self.__age
```

```
    def get_name(self):
```

```
        return self.__name
```

```
    def display_info(self):
```

```
        print(f"Име: {self.__name}\tВъзраст: {self.__age}")
```

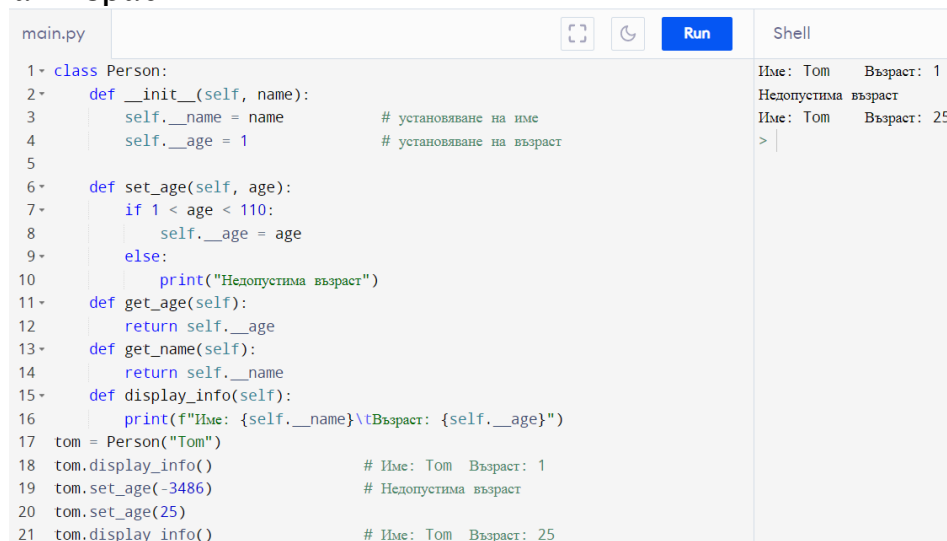
```
tom = Person("Tom")
```

```
tom.display_info()           # Име: Том Възраст: 1
```

```
tom.set_age(-3486)          # Недопустима възраст
```

```
tom.set_age(25)
```

```
tom.display_info()          # Име: Том Възраст: 25
```



```
main.py  Run  Shell
1 class Person:
2     def __init__(self, name):
3         self.__name = name        # установяване на име
4         self.__age = 1           # установяване на възраст
5
6     def set_age(self, age):
7         if 1 < age < 110:
8             self.__age = age
9         else:
10            print("Недопустима възраст")
11
12    def get_age(self):
13        return self.__age
14
15    def get_name(self):
16        return self.__name
17
18    def display_info(self):
19        print(f"Име: {self.__name}\tВъзраст: {self.__age}")
20
21 tom = Person("Tom")
22 tom.display_info()           # Име: Том Възраст: 1
23 tom.set_age(-3486)          # Недопустима възраст
24 tom.set_age(25)
25 tom.display_info()          # Име: Том Възраст: 25
```

Име: Том Възраст: 1  
Недопустима възраст  
Име: Том Възраст: 25  
> |



## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

За да създадете частен атрибут, в началото на името му се поставя двойно тире: `self.__name`. Достъп до такъв атрибут има само от същия клас. Извън този клас няма достъп. Например, ако целим да присвоим стойност на този атрибут няма да се получи:

```
tom.__age = 43
```

Защото в този случай нов атрибут `__age` просто се дефинира динамично, но няма нищо общо със `self.__age`.

И опитът да се получи стойността му ще доведе до грешка по време на изпълнение (ако променливата `__age` не е била предварително дефинирана):

```
print(tom.__age)
```

Въпреки това може да се наложи да се зададе възрастта на потребителя отвън. Използвайки `self` свойство, може да се получи (да се върне) стойността на атрибут:

```
def get_age(self):  
    return self.__age
```

**Този метод също често се нарича `getter` или `Accessor`.**

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Друго свойство определено за промяна на възрастта е:

```
def set_age(self, age):  
    if 1 < age < 110:  
        self.__age = age  
    else:
```

```
        print("Недопустима възраст")
```

**Този метод се нарича още setter или мутатор.** Тук вече според условията, може да се реши дали да се нулира възрастта. Не е необходимо създаване на такава двойка свойства за всеки частен атрибут. Така че в примера по-горе може да се зададе името на човека само от конструктора. И за получаване е дефиниран методът `get_name`.

### Пояснения към свойствата

По-горе беше разгледано как да се създадат свойства. Но Python има и друг, по-елегантен начин за дефиниране на свойства. Този метод включва използване на пояснения, които се предшестват от символа `@`. За да се създаде свойство **getter**, анотацията **@property** се поставя над свойството. За да се създаде свойство за настройка, анотацията **getter\_property\_name.setter** е зададена над свойството.

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Нека се пренапише класа Person, като се използват анотации:

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.__name = name # установяване на име
```

```
        self.__age = 1     # установяване на възраст
```

```
@property
```

```
def age(self):
```

```
    return self.__age
```

```
@age.setter
```

```
def age(self, age):
```

```
    if 1 < age < 110:
```

```
        self.__age = age
```

```
    else:
```

```
        print("Недопустима възраст")
```

```
@property
```

```
def name(self):
```

```
    return self.__name
```

```
def display_info(self):
```

```
    print(f'Име: {self.__name}\tВъзраст: {self.__age}')
```

```
tom = Person("Tom")
```

```
tom.display_info() # Име: Том Възраст: 1
```

```
tom.age = -3486    # Недопустима възраст
```

```
print(tom.age)     # 1
```

```
tom.age = 36
```

```
tom.display_info() # Име: Том Възраст: 36
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
tom = Person("Tom")

tom.display_info() # Име: Tom Възраст: 1
tom.age = -3486    # Недопустима възраст
print(tom.age)     # 1
tom.age = 36
tom.display_info() # Име: Tom Възраст: 36
```

Първо, трябва да се има предвид, че свойството на `setter` е дефинирано след свойството `getter`. Второ, и `setter`, и `getter` се наричат едно и също - `възраст`. Тъй като `getter` се нарича `възраст`, анотацията се задава над `setter` `@age.setter`. След това се отнасяме както към `getter`, така и към `setter` чрез израза `tom.age`.

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

main.py



Run

Shell

```
1 class Person:
2     def __init__(self, name):
3         self.__name = name # установяване на име
4         self.__age = 1     # установяване на възраст
5     @property
6     def age(self):
7         return self.__age
8     @age.setter
9     def age(self, age):
10        if 1 < age < 110:
11            self.__age = age
12        else:
13            print("Недопустима възраст")
14    @property
15    def name(self):
16        return self.__name
17    def display_info(self):
18        print(f"Име: {self.__name}\tВъзраст: {self.__age}")
19    tom = Person("Tom")
20    tom.display_info() # Име: Том Възраст: 1
21    tom.age = -3486    # Недопустима възраст
22    print(tom.age)     # 1
23    tom.age = 36
24    tom.display_info() # Име: Том Възраст: 36
```

```
Име: Том    Възраст: 1
Недопустима възраст
1
Име: Том    Възраст: 36
> |
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Наследство

Наследяването позволява да се създаде нов клас на базата на съществуващ клас. Заедно с капсулирането, наследяването е един от крайъгълните камъни на ООП. Ключовите понятия за наследяване са подклас и суперклас. Подкласът наследява всички публични атрибути и методи от суперкласа. Суперкласът се нарича още базов (базов клас) или родител (родителски клас), а подкласът - производен (производен клас) или дъщерен (детски клас). Синтаксисът за наследяване на клас е както следва:

class подклас (суперклас):

    методи\_на\_подкласа

**Например, класът Person:**

class Person:

```
def __init__(self, name):
    self.__name = name    # име
```

```
@property
def name(self):
    return self.__name
```

```
def display_info(self):
    print(f"Name: {self.__name} ")
```

Да се предположи, че има нужда от създаване на клас Employee, който описва работата на служител в предприятие. Може да се създаде нов клас от нулата, какъвто е клас Employee:

class Employee:

```
def __init__(self, name):
    self.__name = name    # име на работника
```

```
@property
def name(self):
    return self.__name
```

```
def display_info(self):
    print(f"Name: {self.__name} ")
```

```
def work(self):
    print(f"{self.name} works")
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Въпреки това, класът Employee може да има същите атрибути и методи като класа Person, тъй като служителът е човек. Така че в класът Employee е добавен само методът works, останалата част от кода повтаря функционалността на класа Person. Но за да не се дублира функционалността на един клас в друг, в този случай е по-добре да се използва наследяване. Ако се наследи класа Employee от класа Person реализацията ще е:

```
class Person:
```

```
    def __init__(self, name):
        self.__name = name        # име
```

```
    @property
    def name(self):
        return self.__name
```

```
    def display_info(self):
        print(f'Name: {self.__name} ")
```

```
class Employee(Person):
    def work(self):
        print(f'{self.name} works")

tom = Employee("Tom")
print(tom.name)           # Tom
tom.display_info()        # Name: Tom
tom.work()                 # Tom works
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Класът `Employee` напълно поема функционалността на класа `Person`, като добавя само `work()`. Съответно, когато се създава обект `Employee`, може да се използва конструктора, наследен от `Person`:

```
tom = Employee("Tom")
```

Освен това може да се получи достъп до наследени атрибути/свойства и методи:

```
print(tom.name)      # Tom
tom.display_info()   # Name: Tom
```

Трябва да се има предвид, че `Employee` **НЯМА** частни атрибути от тип `__name`. Например, **НЕ** може да получи достъп до частния атрибут в работния метод `self.__name`:

```
def work(self):
    print(f'{self.__name} works')    # ! Error
```

```
class Employee(Person):
```

```
    def work(self):
        print(f'{self.name} works')
```

```
tom = Employee("Tom")
print(tom.name)          # Tom
tom.display_info()       # Name: Tom
tom.work()               # Tom works
```



## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
main.py  [Icons] [Run] Shell

1 ▾ class Person:
2 ▾     def __init__(self, name):
3     |         self.__name = name           # име
4     | @property
5 ▾     def name(self):
6     |         return self.__name
7 ▾     def display_info(self):
8     |         print(f"Name: {self.__name} ")
9 ▾ class Employee:
10 ▾     def __init__(self, name):
11     |         self.__name = name           # име на работника
12     | @property
13 ▾     def name(self):
14     |         return self.__name
15 ▾     def display_info(self):
16     |         print(f"Name: {self.__name} ")
17 ▾     def work(self):
18     |         print(f"{self.name} works")
19 ▾ class Employee(Person):
20 ▾     def work(self):
21     |         print(f"{self.name} works")
22 tom = Employee("Tom")
23 print(tom.name)           # Tom
24 tom.display_info()        # Name: Tom
25 tom.work()               # Tom works
```

Tom  
Name: Tom  
Tom works  
> |

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Множествено наследяване

Една от отличителните черти на езика Python е поддръжката за множествено наследяване, тоест един клас може да бъде наследен от няколко класа:

```
# клас Employee
class Employee:
    def work(self):
        print("Employee works")
```

```
# клас Student
class Student:
    def study(self):
        print("Student studies")
```

#WorkingStudent наследява класовете Employee и Student

```
class WorkingStudent(Employee, Student):
    pass
```

```
# клас Working Student
tom = WorkingStudent()
tom.work()          # Employee works
tom.study()         # Student studies
```

| main.py                                                                                                                                                                                                                                                                                                                                                                                                                                                | Run | Shell                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--------------------------------------------------|
| <pre>1 #клас Employee 2 class Employee: 3     def work(self): 4         print("Employee works") 5 #клас Student 6 class Student: 7     def study(self): 8         print("Student studies") 9 #WorkingStudent наследява класовете Employee и Student 10 class WorkingStudent(Employee, Student): 11     pass 12 # клас Working Student 13 tom = WorkingStudent() 14 tom.work()          # Employee works 15 tom.study()         # Student studies</pre> |     | <pre>Employee works Student studies &gt;  </pre> |

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Той дефинира клас Employee, който представлява служител на фирмата, и студентски клас, който представлява Student. Класът WorkingStudent, който представлява работещ студент, не дефинира никаква функционалност, така че дефинира изявление за преминаване. Класът WorkingStudent просто наследява функционалност от двата класа Employee и Student. Съответно могат да бъдат извикани методите на двата класа върху обект от този клас. В същото време наследените класове могат да бъдат по-сложни по функционалност, например:

```
class Employee:
    def __init__(self, name):
        self.__name = name
    @property
    def name(self):
        return self.__name
    def work(self):
        print(f'{self.name} works')
```

```
class Student:
    def __init__(self, name):
        self.__name = name
    @property
    def name(self):
        return self.__name
    def study(self):
        print(f'{self.name} studies')
class WorkingStudent(Employee, Student):
    pass
tom = WorkingStudent("Tom")
tom.work()           # Tom works
tom.study()          # Tom studies
def study(self):
    print(f'{self.name} studies')
class WorkingStudent(Employee, Student):
    pass
tom = WorkingStudent("Tom")
tom.work()           # Tom works
tom.study()          # Tom studies
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

main.py



Run

Shell

```
1 class Employee:
2     def __init__(self, name):
3         self.__name = name
4     @property
5     def name(self):
6         return self.__name
7     def work(self):
8         print(f"{self.name} works")
9 class Student:
10    def __init__(self, name):
11        self.__name = name
12    @property
13    def name(self):
14        return self.__name
15    def study(self):
16        print(f"{self.name} studies")
17 class WorkingStudent(Employee, Student):
18     pass
19 tom = WorkingStudent("Tom")
20 tom.work()           # Tom works
21 tom.study()          # Tom studies
22 def study(self):
23     print(f"{self.name} studies")
24 class WorkingStudent(Employee, Student):
25     pass
26 tom = WorkingStudent("Tom")
```

```
^ Tom works
Tom studies
Tom works
Tom studies
> |
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### Предефиниране на функционалността на базовия клас

Класът Employee може да възприеме функционалността на класа Person:

```
class Person:
```

```
    def __init__(self, name):
        self.__name = name          # име
```

```
    @property
    def name(self):
        return self.__name
```

```
    def display_info(self):
        print(f"Name: {self.__name} ")
```

```
class Employee(Person):
```

```
    def work(self):
        print(f"{self.name} works")
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Но какво ще стане, ако трябва да се промени нещо от тази функционалност? Например да се добави нов атрибут към служител чрез конструктора, който ще съхранява компанията, в която работи, или да се промени реализацията на метода `display_info`. Python позволява да замени функционалността на базов клас. Например, нека се променят класовете по следния начин:

```
class Person:
```

```
    def __init__(self, name):
        self.__name = name    # име
```

```
    @property
    def name(self):
        return self.__name
```

```
    def display_info(self):
        print(f'Name: {self.__name}')
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
class Employee(Person):
```

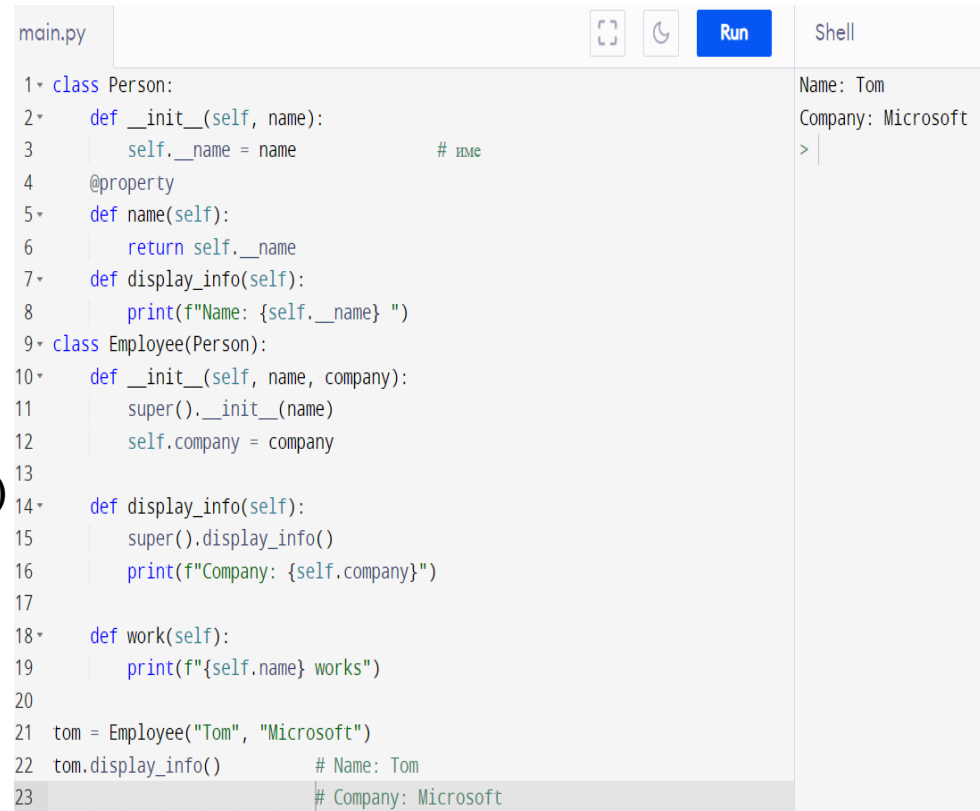
```
    def __init__(self, name, company):
        super().__init__(name)
        self.company = company
```

```
    def display_info(self):
        super().display_info()
        print(f"Company: {self.company}")
```

```
    def work(self):
        print(f"{self.name} works")
```

```
tom = Employee("Tom", "Microsoft")
```

```
tom.display_info()           # Name: Tom
                             # Company: Microsoft
```



```
main.py  Run Shell
1 class Person:
2     def __init__(self, name):
3         self.__name = name          # name
4     @property
5     def name(self):
6         return self.__name
7     def display_info(self):
8         print(f"Name: {self.__name} ")
9 class Employee(Person):
10    def __init__(self, name, company):
11        super().__init__(name)
12        self.company = company
13
14    def display_info(self):
15        super().display_info()
16        print(f"Company: {self.company}")
17
18    def work(self):
19        print(f"{self.name} works")
20
21 tom = Employee("Tom", "Microsoft")
22 tom.display_info()          # Name: Tom
23                             # Company: Microsoft
```

Shell

```
Name: Tom
Company: Microsoft
>
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тук се добавя нов атрибут към класа на служителя - `self.company`, който се съхранява от компанията на служителя. Съответно методът `__init__()` приема три параметъра: вторият за задаване на името и третия за настройка на компанията. Но ако конструкторът е дефиниран в базовия клас с помощта на метода `__init__` и е необходимо да се промени логиката на конструктора в производния клас, тогава в конструктора на производния клас трябва да се извика конструктора на базовия клас. Т.е. в конструктора `Employee` трябва да се извика конструктора на клас `Person`.

Изразът `super()` се използва за препращане към базовия клас. И така, в конструктора `Employee` се извършва извикването:

```
super().__init__(name)
```

Този израз представлява извикване към конструктора на класа `Person`, на който се предава името на работника. И това е логично. В крайна сметка името на работника е зададено в конструктора на класа `Person`. В самия конструктор `Employee` се задава само свойството на компанията. Освен това в класа `Employee`, методът `display_info()` е отменен, като към него се добавя продукцията на компанията на служителя. Методът може да се дефинира по следния начин:



## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
def display_info(self):  
    print(f'Name: {self.name}')  
    print(f'Company: {self.company}')
```

Тогава изходният ред за име ще повтори кода от класа Person. Ако тази част от кода съвпада с метода от класа Person, тогава няма смисъл да се повтаря, така че отново, използвайки израза `super()`, става обръщане към реализацията на метода `display_info` в класа Person:

```
def display_info(self):  
    super().display_info()      # обръщение към метод display_info в клас Person  
    print(f'Company: {self.company}')
```

След това може да се извика конструктора `Employee`, за да се създаде обект от този клас и да се извика метода `display_info`:

```
tom = Employee("Tom", "Microsoft")  
tom.display_info()
```

**Конзолен изход на програмата:**

Name: Tom

Company: Microsoft

## **Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект**

### **Проверка на типа на обекта**

При работа с обекти може да се наложи извършване на определени операции в зависимост от вида им. А с вградената функция `isinstance()` може да провери типа на обекта. Тази функция приема два параметъра:  
`isinstance(object, type)`

Първият параметър представлява обекта, а вторият параметър представлява типа, за който ще се тества. Ако обектът представлява посочения тип, тогава функцията връща `True`. Например, ако се разгледа следната йерархия на класове лице-служител/студент:

**Тема 4\_2.** Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
class Person:
    def __init__(self, name):
        self.__name = name        # име

    @property
    def name(self):
        return self.__name

    def do_nothing(self):
        print(f'{self.name} does nothing')

# клас работник
class Employee(Person):
    def work(self):
        print(f'{self.name} works')

# клас студент
class Student(Person):
    def study(self):
        print(f'{self.name} studies')
```

```
def act(person):
    if isinstance(person, Student):
        person.study()
    elif isinstance(person, Employee):
        person.work()
    elif isinstance(person, Person):
        person.do_nothing()
```

```
tom = Employee("Tom")
bob = Student("Bob")
sam = Person("Sam")
```

```
act(tom)           # Tom works
act(bob)           # Bob studies
act(sam)           # Sam does nothing
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

main.py



Run

Shell

```
1 class Person:
2     def __init__(self, name):
3         self.__name = name          # име
4     @property
5     def name(self):
6         return self.__name
7     def do_nothing(self):
8         print(f"{self.name} does nothing")
9 class Employee(Person): # клас работник
10    def work(self):
11        print(f"{self.name} works")
12 class Student(Person): # клас студент
13    def study(self):
14        print(f"{self.name} studies")
15 def act(person):
16     if isinstance(person, Student):
17         person.study()
18     elif isinstance(person, Employee):
19         person.work()
20     elif isinstance(person, Person):
21         person.do_nothing()
22 tom = Employee("Tom")
23 bob = Student("Bob")
24 sam = Person("Sam")
25 act(tom)                # Tom works
26 act(bob)                # Bob studies
```

```
^ Tom works
Bob studies
Sam does nothing
> |
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тук класът `Employee` дефинира метода `work()`, а класът `Student` дефинира метода на обучение. Той също така дефинира функция `ast`, която проверява с функцията `isinstance` дали параметърът `person` представлява определен тип и в зависимост от резултатите от проверката извиква конкретен метод на обекта.

### Атрибути на класа и статични методи атрибути на класа

В допълнение към атрибутите на обекти в клас, могат да се дефинират атрибути на клас. Такива атрибути се дефинират като променливи на ниво клас. Например:

```
class Person:
    type = "Person"
    description = "Describes a person"

print(Person.type)           # Person
print(Person.description)    # Describes a person

Person.type = "Class Person"
print(Person.type)           # Class Person
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тук в класа `Person` са дефинирани два атрибута: `type`, който съхранява името на класа, и `description`, който съхранява описанието на класа. За да се позовем на атрибутите на клас, може да се използва името на класа, например: `Person.type`, и, подобно на атрибутите на обект, могат да се получат и променят техните стойности.

**Подобни атрибути са общи за всички обекти от класа:**

```
class Person:
```

```
    type = "Person"
    def __init__(self, name):
        self.name = name
```

```
tom = Person("Tom")
```

```
bob = Person("Bob")
```

```
print(tom.type)
```

```
print(bob.type)
```

main.py

```
1 class Person:
2     type = "Person"
3     description = "Describes a person"
4     print(Person.type)           # Person
5     print(Person.description)    # Describes a person
6     Person.type = "Class Person"
7     print(Person.type)           # Class Person
```



Run

Shell

```
Person
Describes a person
Class Person
> |
```

# изменяме атрибута на класа

```
Person.type = "Class Person"
```

```
print(tom.type)
```

```
print(bob.type)
```

```
# Person
```

```
# Person
```

```
# Class Person
```

```
# Class Person
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Атрибутите на класа могат да се използват за ситуации, в които трябва да се дефинират някои общи данни за всички обекти. Например:

class Person:

    default\_name = "Undefined"

    def \_\_init\_\_(self, name):

        if name:

            self.name = name

        else:

            self.name = Person.default\_name

tom = Person("Tom")

bob = Person("")

print(tom.name)               # Tom

print(bob.name)               # Undefined



```
main.py
1 class Person:
2     default_name = "Undefined"
3     def __init__(self, name):
4         if name:
5             self.name = name
6         else:
7             self.name = Person.default_name
8     tom = Person("Tom")
9     bob = Person("")
10    print(tom.name)           # Tom
11    print(bob.name)          # Undefined
```

Tom  
Undefined  
> |

В този случай атрибутът default\_name съхранява името по подразбиране. И ако празен низ за името се предаде на конструктора, тогава стойността на атрибута на класа default\_name се предава на атрибута name. За обръщение към атрибут на клас в метод, може да се използва името на класа:

self.name = Person.default\_name

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

### атрибут на класа

**Възможно е атрибут на клас и атрибут на обект да имат едно и също име. Ако стойността не е зададена за атрибут на обект в кода, тогава стойността на атрибута на класа може да се използва за него:**

```
class Person:
    name = "Undefined"

    def print_name(self):
        print(self.name)

tom = Person()
bob = Person()
tom.print_name()           # Undefined
bob.print_name()           # Undefined

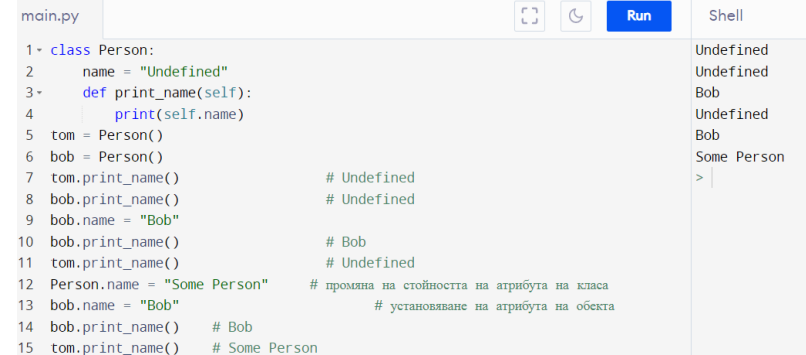
bob.name = "Bob"
bob.print_name()           # Bob
tom.print_name()           # Undefined
```



## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Тук методът `print_name` използва атрибута `name` на обекта, но никъде в кода не е зададен този атрибут. Но на ниво клас атрибутът `name` е зададен. Следователно, първия път, когато методът `print_name` бъде извикан, той ще използва стойността на атрибута на класа:

```
tom = Person()
bob = Person()
tom.print_name()           # Undefined
bob.print_name()           # Undefined
```



The screenshot shows a Python IDE with a file named `main.py`. The code in the editor is as follows:

```
1- class Person:
2-     name = "Undefined"
3-     def print_name(self):
4-         print(self.name)
5- tom = Person()
6- bob = Person()
7- tom.print_name()           # Undefined
8- bob.print_name()           # Undefined
9- bob.name = "Bob"
10 bob.print_name()           # Bob
11 tom.print_name()           # Undefined
12 Person.name = "Some Person" # промяна на стойността на атрибута на класа
13 bob.name = "Bob"           # установяване на атрибута на обекта
14 bob.print_name()           # Bob
15 tom.print_name()           # Some Person
```

On the right side of the IDE, there is a 'Shell' window showing the output of the code execution:

```
Undefined
Undefined
Bob
Undefined
Bob
Some Person
>
```

След това обаче може да се промени атрибута `set` на обекта:

```
bob.name = "Bob"
bob.print_name()           # Bob
tom.print_name()           # Undefined
```

Освен това вторият обект - `tom` ще продължи да използва атрибута клас. И ако се промени атрибута на класа, стойността `tom.name` също ще се промени:

```
tom = Person()
bob = Person()
tom.print_name()           # Undefined
bob.print_name()           # Undefined
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
Person.name = "Some Person"    # промяна на стойността на атрибута на класа
bob.name = "Bob"               # установяване на атрибута на обекта
bob.print_name()               # Bob
tom.print_name()               # Some Person
```

### Статични методи

В допълнение към обикновените методи, един клас може да дефинира статични методи. Такива методи се предшестват от анотацията `@staticmethod` и се отнасят до класа като цяло. Статичните методи обикновено дефинират поведение, което е независимо от конкретен обект:

```
class Person:
    __type = "Person"

    @staticmethod
    def print_type():
        print(Person.__type)

Person.print_type()    # Person – достъп до статичен метод чрез името на класа

tom = Person()
tom.print_type()       # Person – обръщение към статичен метод чрез името на
                        # обекта
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

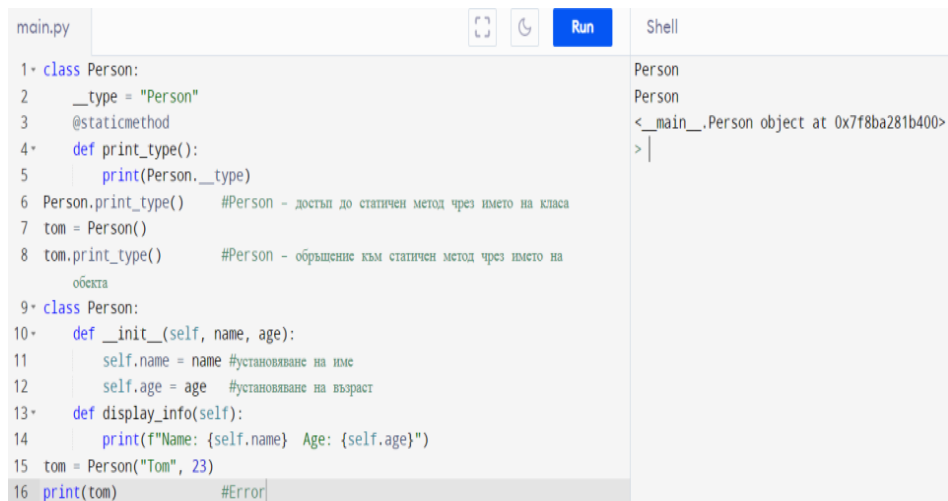
```
class Person:
    def __init__(self, name, age):
        self.name = name          # установяване на име
        self.age = age            # установяване на възраст
```

```
    def display_info(self):
        print(f"Name: {self.name} Age: {self.age}")
```

```
tom = Person("Tom", 23)
print(tom)
```

Когато се стартира, програмата се показва следното:

<\_\_main\_\_.Person обект на 0x10a63dc00>



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```
1 class Person:
2     __type = "Person"
3     @staticmethod
4     def print_type():
5         print(Person.__type)
6 Person.print_type() #Person - достъп до статичен метод чрез името на класа
7 tom = Person()
8 tom.print_type()   #Person - обръщение към статичен метод чрез името на
                     #обекта
9 class Person:
10     def __init__(self, name, age):
11         self.name = name #установяване на име
12         self.age = age   #установяване на възраст
13     def display_info(self):
14         print(f"Name: {self.name} Age: {self.age}")
15 tom = Person("Tom", 23)
16 print(tom)           #Error
```

On the right side, there is a 'Shell' window showing the output of the program:

```
Person
Person
<__main__.Person object at 0x7f8ba281b400>
>
```

The 'Run' button is visible at the top right of the editor area.

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Това не е много информативна информация за обекта. Разбира се, може да заобиколи това, като се дефинира допълнителен метод в класа Person, който показва данните на обекта - в примера по-горе това е методът display\_info. Но има и друг изход – дефиниране на метода \_\_str\_\_() в класа Person (две долни черти от всяка страна):

```
class Person:
    def __init__(self, name, age):
        self.name = name    # установяване на име
        self.age = age      # установяване на възраст

    def display_info(self):
        print(self)
        # print(self.__str__())    # или по този начин

    def __str__(self):
        return f'Name: {self.name} Age: {self.age}'

tom = Person("Tom", 23)
print(tom)                # Name: Tom Age: 23
tom.display_info()        # Name: Tom Age: 23
```

## Тема 4\_2. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Методът `__str__` трябва да върне низ. И в този случай се връща основната информация за лицето. Ако трябва да се използва тази информация в други методи на класа, тогава може да се използва израза `self.__str__()`

Изходът на конзолата ще бъде различен:

Name: Tom Age: 23Name: Tom Age: 23

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                             |                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| main.py                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |    | Shell                                           |
| <pre>1 class Person: 2     def __init__(self, name, age): 3         self.name = name          # установяване на име 4         self.age = age            # установяване на възраст 5 6     def display_info(self): 7         print(self) 8         # print(self.__str__())   # или по този начин 9 10    def __str__(self): 11        return f"Name: {self.name} Age: {self.age}" 12 13 tom = Person("Tom", 23) 14 print(tom)                        # Name: Tom Age: 23 15 tom.display_info()               # Name: Tom Age: 23</pre> |                                                                                                                                                                                                                                                             | <pre>Name: Tom Age: Name: Tom Age: &gt;  </pre> |

## **Използвана литература:**

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009